SUPPORTING DOCUMENT FOR REAL-TIME DRIVER DROWSINESS DETECTION USING DEEP LEARNING-BASED COMPUTER VISION

Saikiran Damalla

Student id: B00979424

Course id: COM748

Computer science and Technology

University: Ulster University

London, United Kingdom

Damalla-S@ulster.ac.uk

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for their valuable guidance and continuous support throughout this project. I also thank the faculty of the Computer Science and Technology department at Ulster University for providing the necessary resources and support.

I am grateful to the developers of the Ultralytics YOLO framework for their tools and documentation, which made this work possible. Finally, I would like to thank my family and friends for their encouragement and support.

1. INTRODUCTION

The main research objective report on Real-Time Driver Drowsiness Detection using Deep Learning-Based Computer Vision is enhanced by this companion document. A few implementation details, technological choices, and system-level explanations could not be fully presented due to space limits in the main paper. This document's goal is to give a thorough explanation of the system, covering dataset preparation, annotation, model building, training procedures, and evaluation methods. Additionally, it explains why the YOLO-based method was chosen for real-time driver drowsiness monitoring.

Improving road safety requires early detection of driver drowsiness, which is a major contributing factor to accidents. Conventional methods of identifying sleepiness depended on physiological cues like electrocardiograms (ECG) and electroencephalograms (EEG). These techniques can yield precise results, but they are unsuitable for practical applications due to their intrusiveness and need for specialised sensors. Research has moved toward non-intrusive computer vision-based methods that use cameras to analyse facial traits including yawning, eye closure, and head posture to get around these restrictions. Convolutional Neural Networks (CNNs) have greatly increased the accuracy of feature extraction and identification thanks to developments in deep learning [6], [8]. Nevertheless, CNN-based techniques require a lot of processing power and aren't necessarily appropriate for real-time implementation.

The YOLO (You Only Look Once) object detection framework, which is popular for real-time applications because of its high speed and efficiency, is employed in this work to address these issues [1], [2]. YOLO enables quick and precise detection by carrying out both localisation and classification in a single forward pass. This project uses the Ultralytics platform to create a customised YOLOv26 model that can identify driver tiredness in real time. The system's practical implementation in driver monitoring systems is made possible by its architecture, which strikes a compromise between accuracy, computational economy, and low latency. Unlike alcohol, fatigue is hard to detect and frequently goes unreported, making it very harmful.

According to UK road safety data, driver fatigue contributes to around 10–20% of road accidents and up to 25% of fatal crashes, with over 1,300 fatigue-related collisions reported annually [22],[23].

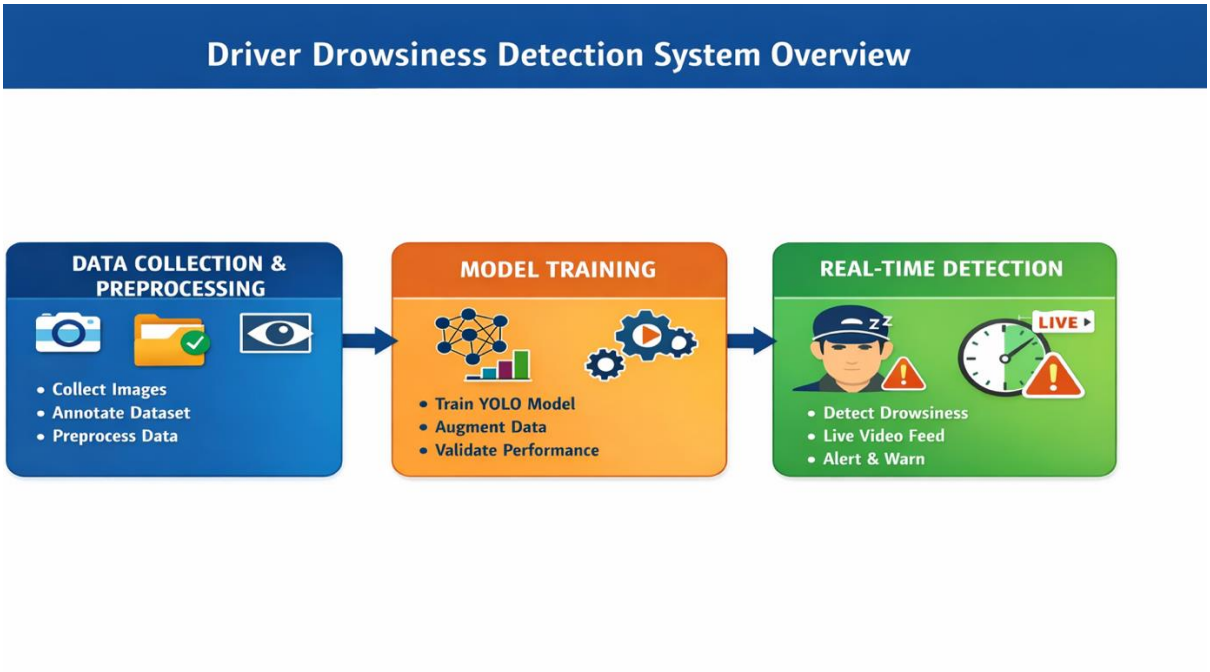

Figure 1: System Overview Diagram

2. EXTENDED LITERATURE REVIEW

2.1. Traditional Methods

Conventional techniques for detecting driver drowsiness mostly use manually created features that are taken from facial landmarks. Eye Aspect Ratio (EAR), Percentage of Eye Closure (PERCLOS), blink rate analysis, and head posture estimate are common methods. By examining the geometric correlations between face characteristics, especially the eyes and head position, these techniques determine the driver's level of awareness. While PERCLOS assesses the percentage of time the eyes remain closed over a given period, it is a reliable indication of weariness [10], [11]. EAR calculates the ratio of distances between eye landmarks to evaluate eye closure. These methods are appropriate for systems with minimal resources and are computationally light.

Nevertheless, conventional approaches have a few drawbacks despite their ease of use. They are extremely sensitive to environmental factors like shadows, lighting changes, and camera placement. Occlusions brought on by masks, glasses, or head movements can also drastically lower detection accuracy. These methods are not flexible and do not perform well in complex real-world driving situations because they are rule-based.

2.2. Deep Learning-Based Methods (CNN)

Deep learning methods, especially Convolutional Neural Networks (CNNs), have greatly enhanced sleepiness detection systems with the development of artificial intelligence. CNN models eliminate the need for manual feature extraction by automatically learning complicated properties from visual data, such as yawning, eye closure, and facial expressions.
Strong performance in visual recognition tests and greatly enhanced feature extraction capabilities have been shown by deep CNN architectures [6], [8]. CNN-based techniques are more resilient to changes in lighting, backdrop, and facial position than conventional techniques.

CNN-based techniques have significant disadvantages despite these benefits. For training and inference, they need high-performance technology like GPUs and are computationally costly. Additionally, a lot of CNN models are limited in their capacity to localise certain regions of interest (such the face or eyes) because they are made for image categorisation rather than object recognition. Because of this, they are less appropriate for real-time driver monitoring systems where localisation and speed are crucial.

2.3. YOLO-Based Approaches

Single-stage object detection models like YOLO (You Only Look Once) have drawn a lot of attention to get around the drawbacks of conventional and CNN-based techniques. YOLO enables quick and effective detection by combining object localisation and classification into a single forward pass.
In real-time driver monitoring systems, YOLO-based methods have demonstrated excellent performance [17], [18]. Compared to multi-stage detectors like R-CNN, the model is far faster because it processes the full image in a single pass. YOLO can achieve high speed and good accuracy thanks to this single-stage detection technology [1], [3].

YOLO is especially well suited for applications like driver drowsiness detection, where prompt response is crucial, because of its low latency and real-time capability. It is more efficient than conventional and CNN-based methods for real-time situations because it can simultaneously identify facial characteristics and categorise driving states.

2.4. Research Gap

Deep learning and YOLO-based techniques have made great strides, but there are still a few obstacles to overcome. Many current systems prioritise increasing detection accuracy above taking real-time deployment restrictions and computational economy into account. Furthermore, real-world variances like head motions, illumination changes, and occlusions frequently cause problems for existing models.
Additionally, many studies do not concurrently optimise latency and robustness, which is crucial for realistic driver monitoring systems. This disparity emphasises the necessity for a solution that strikes a compromise between real-time performance, computing efficiency, and accuracy.

This paper suggests a tailored YOLOv26-based driver sleepiness detection system utilising the Ultralytics framework to overcome these drawbacks. To increase robustness under a variety of real-world scenarios while preserving low latency and computational efficiency, the system incorporates improved data augmentation techniques and optimised training methodologies.

The following table X presents a comparison of major approaches:

Approach	Accuracy	Speed	Real-Time Capability	Limitations
EAR / PERCLOS	Medium	High	Limited	Sensitive to lighting, head movement, and occlusion
Machine Learning	Medium	Medium	Limited	Requires manual feature extraction and lacks adaptability
CNN	High	Low	Limited	Computationally expensive and slower inference
YOLO	High	Very High	Excellent	Slight trade-off in localization precision

The comparison presented in Table X is based on findings from previous studies on object detection models [1], [3], [4], [5].

3. SYSTEM DEVELOPMENT LIFE CYCLE

3.1 Workflow

To ensure effective data processing and real-time prediction, the suggested driver drowsiness detection system uses a structured and sequential pipeline. The following steps make up the workflow:

Data Collection → Annotation → Preprocessing → Augmentation → Training → Evaluation → Prediction

Initially, class labels are added to raw image data that has been gathered. After that, the data is enhanced and pre-processed to increase diversity and quality. The YOLO-based model is trained using the processed dataset. Lastly, the trained model is assessed and used to predict driver states in real time (normal or sleepy).

A seamless transition from raw data to a fully functional real-time detection system is ensured by this organised procedure.

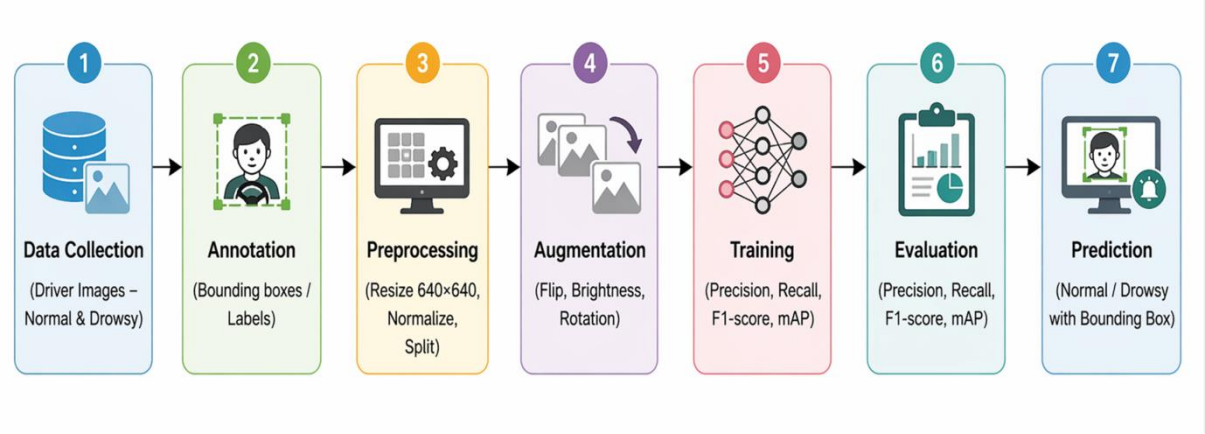

Figure 2: Workflow Diagram

3.2 Dataset Preparation

To train the model effectively, a custom dataset was created consisting of driver images categorized into two classes:

- **Normal (Awake State)**
- **Drowsy (Fatigued State)**

The dataset includes a wide variety of real-world conditions to improve generalization:

- Different lighting conditions (day, night, low light)
- Various head orientations (left, right, tilted)
- Variations in facial expressions and eye states

The dataset was created and annotated manually. Additionally, publicly available datasets and annotation tools were referenced to ensure consistency with object detection formats [4], [5].

To avoid bias, the dataset was balanced across both classes, ensuring equal representation of normal and drowsy samples.

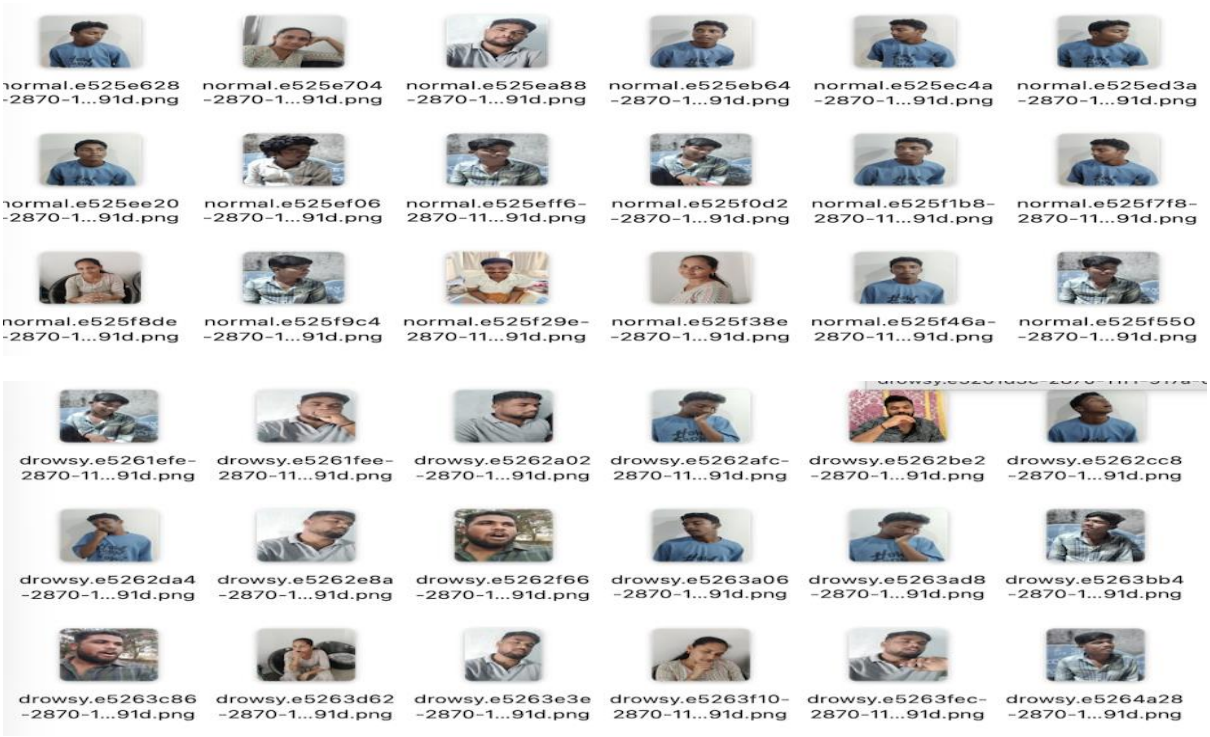

Figure 3: Sample Dataset Images (Normal vs Drowsy)

3.3 Data Annotation

Each image in the dataset was annotated using bounding boxes to identify the region of interest, typically the driver's face. This step is crucial for training the YOLO model, as it enables both:

- **Localization** (detecting where the face is)
- **Classification** (determining whether the driver is normal or drowsy)

Annotations were converted into YOLO format, which includes bounding box coordinates and class labels. Accurate annotation is essential, as incorrect labels can significantly affect model performance.

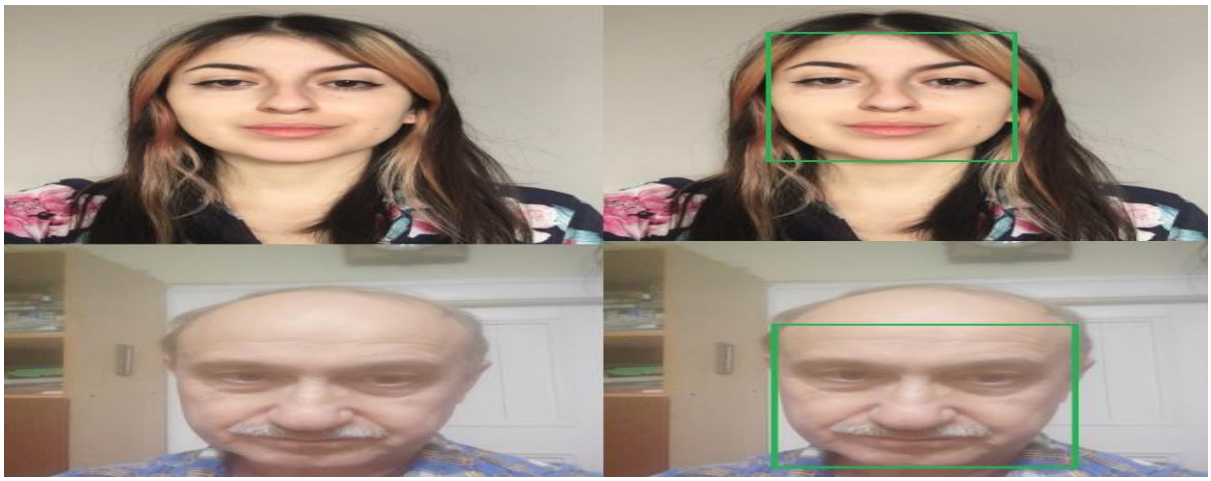

Figure 4: Annotation Example (Bounding Boxes)

3.4 Data Preprocessing

To ensure consistency and improve model performance, several preprocessing steps were applied:

- **Image Resizing:**
  All images were resized to **640 × 640 pixels**, which is the standard input size for YOLO models.
- **Normalization:**
  Pixel values were normalized to improve training stability and convergence.
- **Dataset Splitting:**
  - Training set (used for model learning)
  - Validation set (used for performance monitoring)

These steps help the model generalize effectively to unseen data.

3.5 Data Augmentation

To enhance dataset diversity and improve robustness, data augmentation techniques were applied during training. These transformations simulate real-world variations and reduce overfitting.

The following augmentation techniques were used:

- **Horizontal Flipping:** Simulates different head orientations
- **Brightness Adjustment:** Handles varying lighting conditions
- **Color Transformations:** Improves robustness to color changes
- **Rotation & Contrast Enhancement:** Helps in low-light and tilted scenarios

Data augmentation was practically implemented during training, and sample augmented images were generated to verify transformations such as flipping, brightness adjustment, and rotation. These augmentations significantly improved the model's ability to generalize across different real-world conditions.

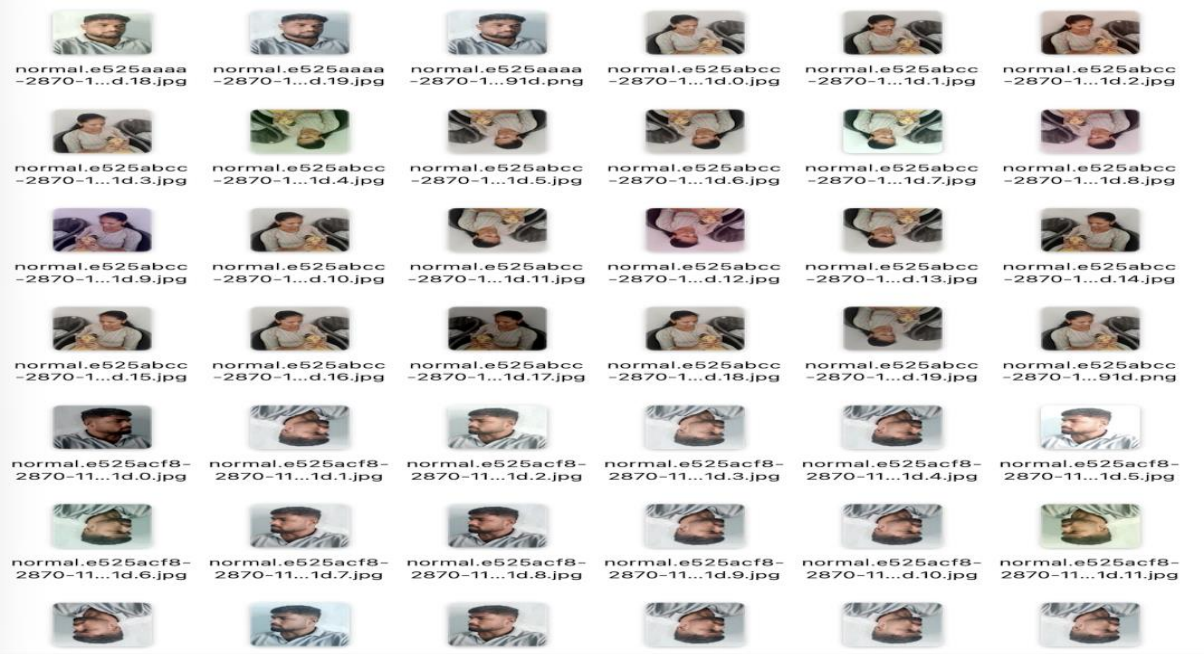

 Figure **5: Augmented Images (Flip, Brightness, Rotation)**

3.6 YOLO Model Development

A single-stage object detection approach called YOLO (You Only Look Once) was created for real-time applications [1]. YOLO is very efficient since it processes the full image in a single forward pass, unlike conventional multi-stage detectors.
This single-stage detection design allows for quick and real-time predictions and low latency for the suggested system [1]. Furthermore, the Ultralytics YOLO framework offers optimised implementations that enhance training efficiency and inference performance even more.
By removing the region proposal steps, the suggested model lowers computing complexity and enables quicker inference appropriate for real-time applications.
In this work, the Ultralytics framework was used to create a customised YOLOv26 model. By enhancing robustness in real-world scenarios and optimising training parameters, the model was specifically designed for driver sleepiness detection.

The model predicts:

- Bounding box coordinates
- Confidence scores
- Class probabilities

This allows simultaneous localization and classification of driver states.

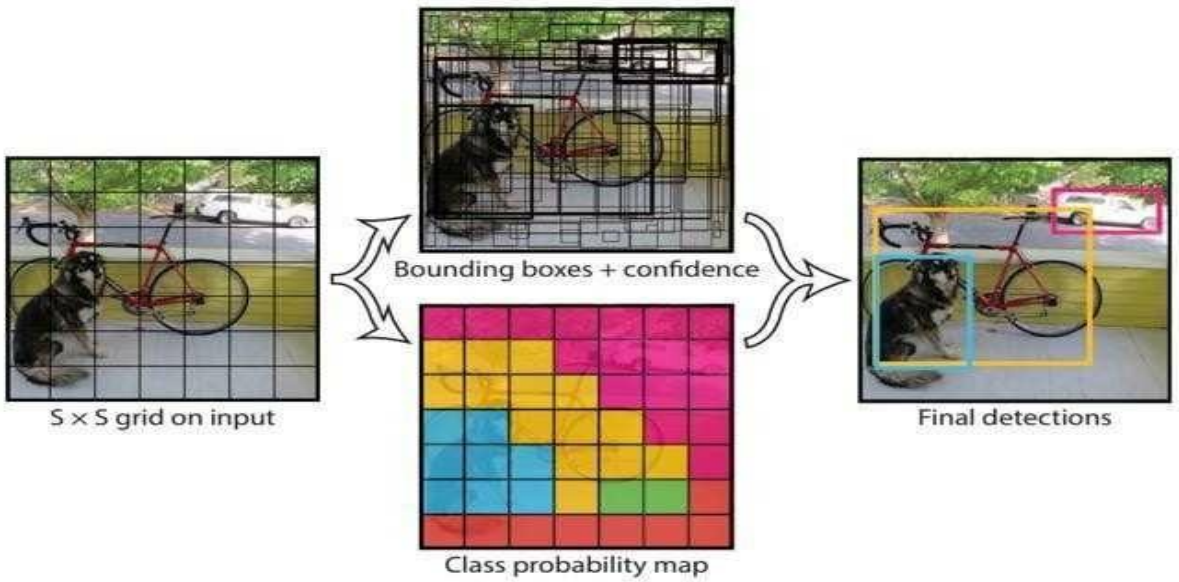

Figure 6: YOLO Architecture Diagram

3.7 Training Process

The YOLOv26 model was trained using a structured training process:

- **Epochs:** 100
- **Pretrained weights:** Used to improve convergence
- **Optimization:** Applied to minimize loss
- **Validation Monitoring:** Used to track performance and avoid overfitting

During training:

- Loss gradually decreased
- Accuracy improved over epochs
- Model learned to distinguish between normal and drowsy states effectively

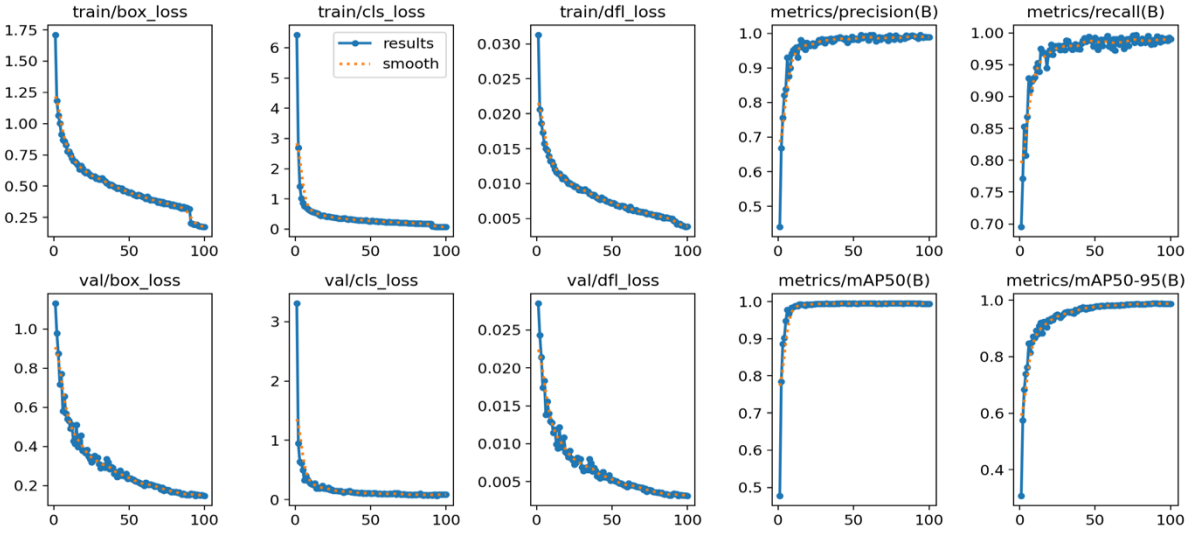

Figure 7: Training Graphs

3.8 Technologies Used

Programming, computer vision, deep learning, and annotation tools are used in the development of the suggested driver drowsiness detection system. Each technology has a distinct function in real-time detection, model training, and dataset preparation.

a. Python

The primary programming language utilised to implement the entire system is Python. Training, preprocessing, and detection scripts are developed with it. Python is appropriate for developing end-to-end deep learning applications since it allows integration with YOLO, OpenCV, and numerical tools.

b. OpenCV

OpenCV is used for real-time image processing and detection.

• Captures frames from webcam or input images
• Processes frames before passing to the model
• Draws bounding boxes and class labels
• Displays real-time detection results

OpenCV enables continuous monitoring of the driver, making the system suitable for real-time applications.

c. Ultralytics YOLO (YOLOv26)

The Ultralytics YOLO framework is used for object detection model development.

• Used for training and inference
• Supports custom dataset training
• Provides optimized real-time detection
• Performs detection in a single forward pass

YOLO is widely used for real-time object detection due to its speed and efficiency [1], [2]. The model processes the entire image in a single forward pass, enabling low-latency detection [1].

In this project, a customized YOLOv26 model is used to classify driver states (normal and drowsy) efficiently.

Import YOLO26

```
[ ]: from ultralytics import YOLO
     import os
     import json
     import uuid
     import numpy as np
     import matplotlib.pyplot as plt

[ ]: dataset_folder = 'dataset'

[ ]: model = YOLO("yolo26n.pt")

[ ]: img = os.path.join('test', 'test1.jpg')
     results = model(img)
     print(results)
```

Figure 8: YOLO Model Training and Initialization

d. LabelMe (Annotation Tool)

LabelMe is used for annotating the dataset.

• Draws bounding boxes on driver faces
• Assigns class labels (Normal / Drowsy)
• Generates annotations for training
• Supports conversion to YOLO format

Accurate annotation is essential for training object detection models effectively [4], [5].

Convert labelme coordinates to YOLO coordinates

```
[ ]: def labelme_to_yolo(data):
    if data['shapes'][0]['label'] == "normal":
        label_class = 0
    else:
        label_class = 1

    image_width = data['imageWidth']
    image_height = data['imageHeight']

    x_min = data['shapes'][0]['points'][0][0]
    x_max = data['shapes'][0]['points'][1][0]
    y_min = data['shapes'][0]['points'][0][1]
    y_max = data['shapes'][0]['points'][1][1]

    x_center = (x_min + x_max) / 2
    y_center = (y_min + y_max) / 2

    bbox_width = x_max - x_min
    bbox_height = y_max - y_min

    return f'{label_class} {x_center/image_width} {y_center/image_height} {bbox_width/image_width} {bbox_height/image_height}'


    #print(x_min, x_max, y_min, y_max)
    #print(data['imageWidth'])
    #print(data['imageHeight'])
```

Figure 9: Annotation and Conversion to YOLO Format

e. NumPy

NumPy is used for numerical operations and handling image data.

• Stores image data as arrays
• Supports preprocessing operations
• Enables efficient mathematical computations

NumPy helps in preparing data for model training.

f. Matplotlib

Matplotlib is used for visualizing model performance.

• Displays training graphs
• Helps analyze model performance
• Supports evaluation visualization

g. Google Colab

Google Colab is used as the training environment.

• Provides GPU support for faster training
• Enables running deep learning models without local setup
• Supports Python and YOLO integration

The YOLOv26 model was trained in this environment to improve training efficiency.

3.9 Code Implementation

The system was implemented using Python and the Ultralytics YOLO framework. The code is divided into three main components:

a. Dataset Configuration

This step involves defining dataset paths, class labels, and configuration files required for training.

- Dataset directory structure
- YAML configuration file
- Class definitions (Normal, Drowsy)

Drowsiness Project.ipynb | dataset.yaml

```yaml
path: dataset/

train: images
val: images

names:
    0: normal
    1: drowsy
```

Figure 10: Dataset Configuration Code

b. Model Training Code

The training process was executed using Ultralytics YOLO commands/scripts. This includes:

- Loading pretrained weights
- Setting epochs and batch size
- Training on custom dataset
- Saving trained model weights

Example operations performed:

- Model initialization
- Training execution
- Performance logging

Training Model

```
[ ]: results = model.train(data="dataset.yaml", epochs=100, imgsz=640)
```

Figure 11: Training Code

c. Prediction Code (Real-Time Implementation)

The trained model was deployed for real-time detection using OpenCV. The system captures video input (webcam or camera) and processes each frame to detect driver state.

Steps involved:

- Capture video frame
- Pass frame to YOLO model
- Detect face and classify state
- Draw bounding boxes and labels
- Display output in real time

This enables continuous monitoring of the driver.

Figure 12: Prediction Code (OpenCV / Webcam)

4. MODEL EVALUATION AND RESULTS

The performance of the proposed YOLO-based driver drowsiness detection system is evaluated using standard object detection metrics. The results demonstrate that the model achieves high accuracy and is suitable for real-time applications.

4.1 Evaluation Metrics

The model is evaluated using the following metrics:

• **Precision** – Measures correctness of positive predictions
• **Recall** – Measures ability to detect drowsy cases
• **F1-score** – Balance between precision and recall
• **Mean Average Precision (mAP)** – Overall detection performance

The obtained results are:

• **F1-score ≈ 0.99**
• **mAP@0.5 ≈ 0.995**

These values indicate that the model performs effectively in distinguishing between normal and drowsy driver states.

4.2 Performance Analysis

The performance of the model is further analysed using evaluation curves and matrices.

a. F1-Score Curve

The F1-score curve shows the balance between precision and recall across different confidence thresholds. A consistently high F1-score indicates stable and reliable performance.

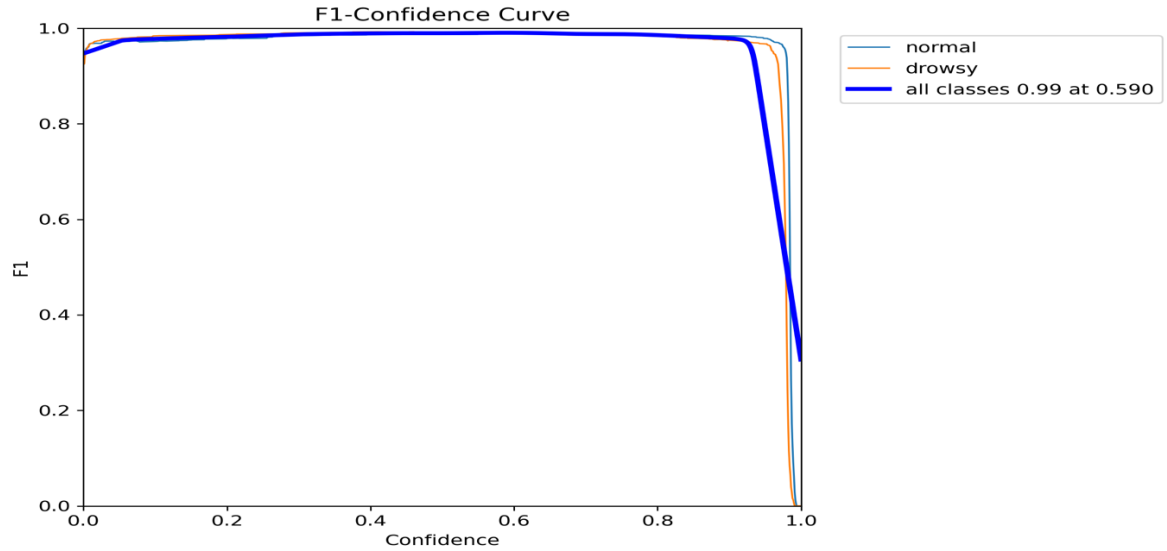

Figure 13: F1-Score Curve

b. Precision Curve

The precision curve represents the model's ability to avoid false positives. High precision values indicate that most predicted drowsy cases are correct.

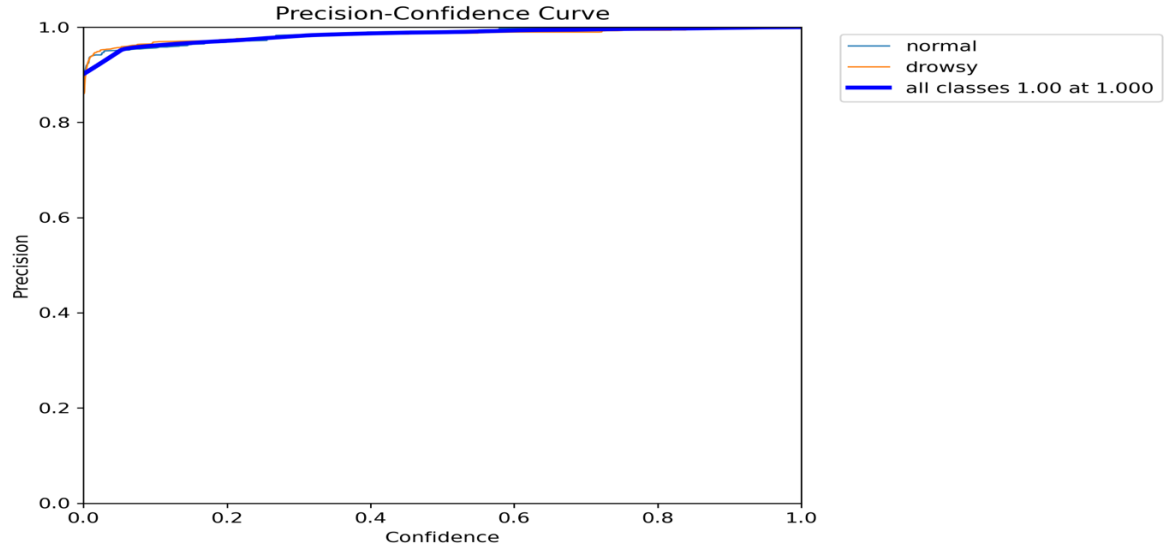

Figure 14: Precision Curve

c. Precision–Recall (PR) Curve

The PR curve illustrates the trade-off between precision and recall. A curve closer to the top-right corner indicates better performance.

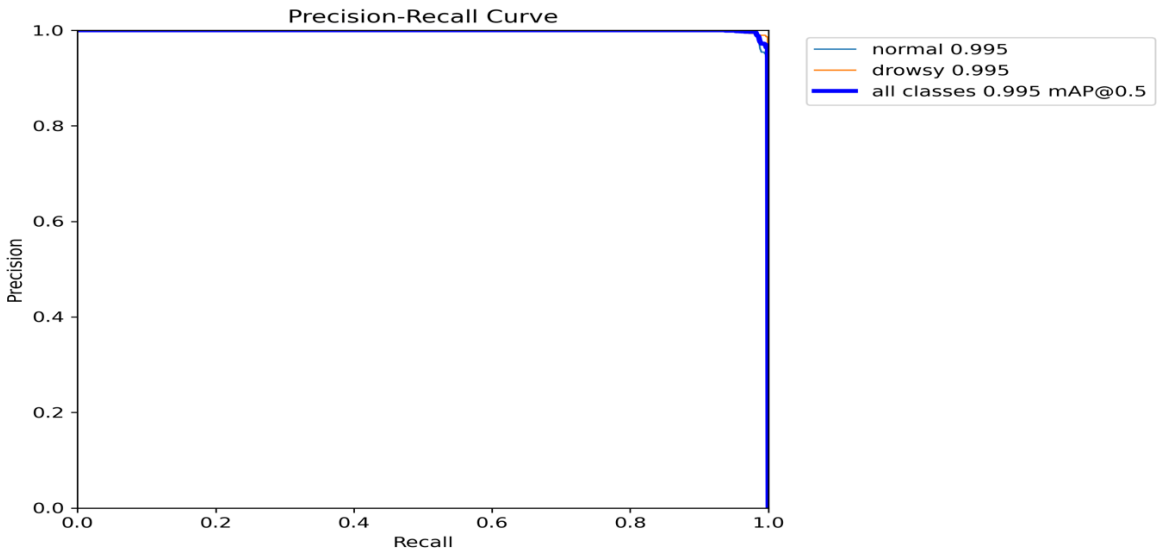

Figure 15: Precision–Recall Curve

d. Confusion Matrix

The confusion matrix provides a detailed view of classification performance.

- Most **normal** and **drowsy** cases are correctly classified
- Very few misclassifications are observed
- Indicates strong generalization capability

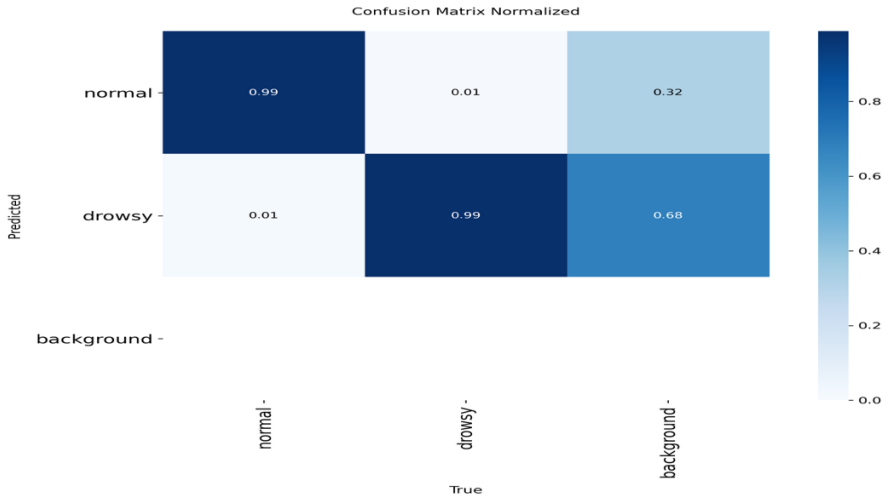

Figure Figure 16: Confusion Matrix (Normalized)

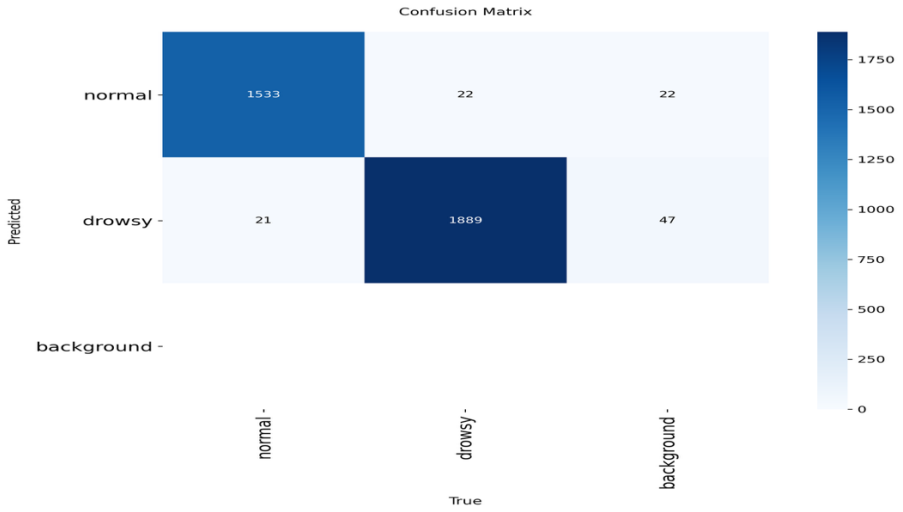

 Figure 17: Confusion Matrix

4.3 Real-Time Performance

One of the key strengths of the proposed system is its real-time detection capability.

The model achieves fast inference due to:

• Single-stage detection architecture of YOLO
• Processing the entire image in a single forward pass
• Absence of region proposal stages

This enables low-latency detection, making the system suitable for real-time applications such as:

• Driver monitoring systems
• Intelligent transportation systems
• Road safety applications

5. CRITICAL APPRAISAL

This experiment shows that real-world deployment of driver drowsiness detection systems requires more than just excellent accuracy. Accuracy, computational efficiency, and real-time performance must all be balanced in realistic applications.
The employment of the YOLO object detection model in place of conventional CNN-based classification techniques is a crucial design choice in this work. YOLO does both localisation and classification in a single forward pass, greatly increasing detection speed and making it appropriate for real-time applications, in contrast to CNN models that only conduct classification [1]. The need for low-latency driver monitoring systems is clearly addressed by this decision.

The usage of structured dataset preparation and data augmentation is another crucial component of this endeavour. To replicate real-world fluctuations, methods including rotation, flipping, and brightness modulation were used. This increases robustness by enhancing the model's capacity to handle various illumination situations, head motions, and facial angles.

The project also highlights that model performance is not solely dependent on architecture. Instead, the following factors play a critical role:

• **Dataset quality** – Diverse and balanced data improves generalization
• **Annotation accuracy** – Correct bounding boxes ensure effective learning
• **Training strategy** – Proper configuration enhances model performance

Additionally, this study sets itself apart by utilising the Ultralytics framework to create a customised YOLOv26 model that is optimised for computational efficiency and accuracy. The system can be deployed in real time because it is made to minimise latency and remove pointless computing stages.
Overall, the experiment shows that a dependable and useful driver sleepiness detection system can be created by combining an effective detection model with appropriate data preparation and optimisation techniques.

6. PROFESSIONAL, ETHICAL, SOCIAL AND SUSTAINABILITY CONSIDERATIONS

The proposed YOLO-based driver drowsiness detection system, while technically effective, involves several important professional, ethical, social, and sustainability considerations that must be carefully addressed for responsible real-world deployment.

6.1 Professional Considerations

From a professional standpoint, the system must ensure high reliability, accuracy, and robustness, as it directly relates to road safety. Since the model is designed for real-time detection using the YOLOv26 architecture, any incorrect prediction—such as failing to detect drowsiness (false negative) or incorrectly flagging an alert (false positive)—can have serious consequences.

To address this, the project incorporates:

- A **structured dataset preparation process** with balanced classes (normal and drowsy)
- **Accurate annotation using bounding boxes**, ensuring proper learning
- **Evaluation using metrics such as F1-score (≈0.99) and mAP (≈0.995)**, demonstrating strong performance
- **Real-time testing using OpenCV-based prediction**, ensuring system responsiveness

Additionally, developers must follow proper software engineering practices such as testing, validation, and documentation to ensure that the system performs reliably under real-world conditions such as lighting variations, head movements, and occlusions.

6.2 Ethical Considerations

The system relies on continuous monitoring of the driver using a camera, which raises significant ethical concerns related to **privacy and data protection**.

Key ethical considerations in this project include:

- **User Consent:**
  Drivers must be informed and give consent before being monitored.
- **Data Privacy:**
  Facial images are sensitive biometric data. The system should avoid unnecessary storage of images or ensure secure storage if required.
- **Compliance with Regulations:**
  The system should comply with data protection laws such as GDPR, especially in regions like the UK where this project is developed.
- **Bias and Fairness:**
  Since the model is trained on a custom dataset, lack of diversity (lighting, skin tone, facial features) can lead to biased predictions.
  To mitigate this:

- Dataset includes variations in lighting and head orientation
- Data augmentation techniques (flip, brightness, rotation) were applied

Ensuring fairness and avoiding discrimination is critical for ethical AI deployment.

6.3 Social Impact

The proposed system has strong positive social implications, particularly in improving **road safety and accident prevention**.

Driver fatigue is one of the major causes of road accidents, as discussed in the introduction section . By enabling real-time detection of drowsiness, the system can:

- Alert drivers before critical situations occur
- Reduce fatigue-related accidents
- Support intelligent transportation systems
- Enhance Advanced Driver Assistance Systems (ADAS)

However, there are also potential concerns:

- **Over-reliance on automation:**
  Drivers may depend too much on the system instead of staying alert.
- **User acceptance:**
  Continuous monitoring may make some users uncomfortable.

Therefore, the system should be designed as a **support tool**, not a replacement for driver responsibility.

6.4 Sustainability Considerations

Sustainability in this project is primarily related to **computational efficiency and resource optimization**.

The use of YOLO provides several sustainability advantages:

- **Single-stage detection architecture** reduces computational load
- **Low latency processing** enables real-time detection without heavy infrastructure
- **Elimination of region proposal stages** reduces unnecessary computation
- Suitable for **edge device deployment**, reducing dependence on high-power servers

This efficient design contributes to:

- Lower energy consumption
- Reduced hardware requirements
- Cost-effective deployment in real-world systems

In addition, by helping prevent accidents, the system contributes to **social and economic sustainability** by reducing:

- Medical and emergency response costs
- Vehicle damage and insurance claims
- Loss of human life

6.5 Summary

All things considered, the suggested YOLO-based driver sleepiness detection system not only exhibits excellent technical performance but also emphasises the significance of responsible AI development. The system can be securely and successfully implemented in real-world driver monitoring applications by addressing professional dependability, ethical issues, social impact, and sustainability.

7. LIMITATIONS

The suggested YOLO-based driver drowsiness detection system has some drawbacks that should be considered despite its excellent performance and high accuracy.
First, the quality and diversity of the dataset have a significant impact on the model's performance. The dataset might not accurately reflect all real-world driving situations, despite efforts to incorporate differences in lighting, head tilt, and facial expressions. The model's capacity to generalise in novel settings may be impacted by this.
Second, under extreme situations like low light, partial face visibility, or occlusions from masks, sunglasses, or head motions, the system may perform less accurately. These elements may affect overall prediction performance and interfere with face detection.

The current model's binary categorisation, which only classifies drivers as normal or sleepy, is another drawback. Since drowsiness develops gradually in real-world situations, the algorithm is unable to identify intermediate levels of exhaustion, which could result in earlier alerts.
Furthermore, no real-world deployment scenario, such as in-car settings with continuous video streams, has been used to evaluate the system. Consequently, elements like shifting camera positions, motion blur, and real-time limitations have not been thoroughly assessed.
Overall, even though the system shows great promise, resolving these issues is crucial to enhancing dependability and guaranteeing successful real-world application.

8. FUTURE ENHANCEMENTS

The suggested YOLO-based driver drowsiness detection system performs well, but there are a few ways to improve its resilience, functionality, and practicality.
The incorporation of real-time video processing via a webcam or in-car camera system is one of the main improvements. This would make it possible to monitor the driver continuously and enable the system to quickly identify tiredness while driving, making deployment more feasible.
The use of an alert system, such as audio or visual alerts, is another significant enhancement. The technology may alert the driver in real time when it detects signs of tiredness, reducing the risk of collisions and enhancing traffic safety.

Binary classification (normal or sleepy) is carried out by the current model. This can be expanded to multi-class identification in further studies, using other signs such head nodding, frequent blinking, and yawning. This would make it possible to identify fatigue levels in a more thorough and early manner.
Additional enhancements can concentrate on improving the system's performance in difficult real-world scenarios, like dimly lit areas, occlusions (like masks or sunglasses), and different camera angles. This can be accomplished by using more sophisticated augmentation techniques and growing the dataset.
The model's implementation on edge devices, like mobile platforms or embedded systems, is another noteworthy improvement. Real-time detection might be possible without requiring a lot of processing power if the model were optimised for lightweight performance, which would make it appropriate for in-car systems.

Additionally, by emphasising the areas of the image the model concentrates on during prediction, Explainable AI techniques like Grad-CAM can increase transparency. This improves the system's interpretability and trustworthiness.
Lastly, testing and validation in real-world driving settings may be a part of future research. This would offer insights for additional optimisation and assist in assessing system reliability under real-world circumstances.

9. CONCLUSION

In this work, deep learning and computer vision techniques were used to construct and develop a YOLO-based driver sleepiness detection system. Using real-time object identification capabilities, the system efficiently divides driver states into normal and sleepy groups.
The model was able to interpret images in a single forward pass because to the effective training and quick inference made possible by the Ultralytics YOLO platform. This guarantees low-latency performance and greatly increases computing efficiency, making the system appropriate for real-time applications. The model's great capacity to detect drowsiness under a variety of settings is demonstrated by the high evaluation metrics, such as F1-score and mAP.

The combination of precise annotation, systematic dataset preparation, and data augmentation strategies, which together improve the model's robustness and generalisation, is a major strength of this work. The suggested system improves performance and speed by combining detection and classification into a single framework, in contrast to conventional methods.
Additionally, the system tackles significant issues found in current approaches, namely the requirement for computational efficiency and real-time performance. The system strikes a balance between accuracy and efficiency by using an optimised training strategy and a customised YOLOv26 model, which qualifies it for practical deployment.

To sum up, the suggested system shows great promise for incorporation into intelligent driver support systems. It demonstrates how deep learning-based computer vision can enhance road safety by facilitating the prompt identification of driver fatigue and assisting with preventive actions in practical situations.

10.REFERENCES

1. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
2. J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” *Proc. IEEE CVPR*, 2017.
3. A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
4. G. Jocher et al., “YOLOv5,” *GitHub Repository, Ultralytics*, 2020.
5. G. Jocher et al., “Ultralytics YOLOv8,” *Ultralytics Documentation*, 2023.
6. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *NeurIPS*, 2012.
7. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *ICLR*, 2015.
8. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *CVPR*, 2016.
9. A. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” 2017.
10. M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *ICML*, 2019.
11. S. Abtahi, B. Hariri, and S. Shirmohammadi, “Driver Drowsiness Monitoring Based on Yawning Detection,” *IEEE International Instrumentation and Measurement Technology Conference*, 2011.
12. T. Soukupová and J. Čech, “Real-Time Eye Blink Detection Using Facial Landmarks,” *Computer Vision Winter Workshop*, 2016.
13. M. S. Hossain et al., “A Real-Time Driver Drowsiness Detection System Using Deep Learning,” *IEEE Access*, 2019.
14. S. Singh and N. Papanikolopoulos, “Monitoring Driver Fatigue Using Facial Analysis Techniques,” *IEEE Intelligent Transportation Systems*, 1999.
15. R. Jabbar et al., “Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques,” *Procedia Computer Science*, 2018.
16. A. Dwivedi, K. Biswaranjan, and A. Sethi, “Drowsy Driver Detection Using Representation Learning,” *IEEE ICCV Workshops*, 2014.

17. H. Park, K. Jung, and H. Chung, “Driver Drowsiness Detection System Based on Feature Representation Learning,” *IEEE Access*, 2018.

18. X. Liu et al., “Real-Time Driver Drowsiness Detection Based on YOLO Algorithm,” *IEEE Access*, 2021.
19. Y. Zhang et al., “Driver Fatigue Detection Based on YOLOv5,” *Journal of Advanced Transportation*, 2022.
20. M. Albadawi et al., “YOLO-Based Real-Time Drowsiness Detection System,” *Sensors*, 2023.
21. S. K. Patel and R. Patel, “Driver Drowsiness Detection Using YOLO and Deep Learning,” *International Journal of Computer Applications*, 2022.
22. RoSPA, “Driver Fatigue and Road Collisions,” 2024.
23. British Safety Council / RoSPA statistics on fatigue-related crashes.
24. S. Wang et al., “Deep Learning for Real-Time Driver Fatigue Detection,” *IEEE Transactions on Intelligent Transportation Systems*, 2020.
25. N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” *CVPR*, 2005.
26. D. King, “Dlib-ml: A Machine Learning Toolkit,” *Journal of Machine Learning Research*, 2009.
27. “Deep Learning-Based Driver Drowsiness Detection System,” *Procedia Computer Science*, 2024.
28. S. Ramzan et al., “A Survey on Driver Drowsiness Detection Systems Using Machine Learning,” *IEEE Access*, 2019.
29. Z. Zhang, “Facial Landmark Detection by Deep Learning,” *IEEE Signal Processing Magazine*, 2016.
30. I.Goodfellow, Y. Bengio, and A. Courville, “Deep Learning,” MIT Press, 2016.
31. R. Girshick, “Fast R-CNN,” *ICCV*, 2015.
32. S. Ren et al., “Faster R-CNN: Towards Real-Time Object Detection,” *NeurIPS*, 2015.

11.CODE AND SUPPLEMENTARY MATERIALS

- Links to GitHub repository

  https://github.com/SaikiranDamalla/DROWSINESS.git

- Jupyter Notebook with full training, evaluation, and inference code

APPENDIX

Appendix A: System Design and Workflow

The overall architecture of the proposed driver drowsiness detection system is illustrated in **Figure 1**, which shows the interaction between data input, processing, model prediction, and output display.

Figure 1: System Overview Diagram

The system follows a structured pipeline from data collection to prediction. This workflow is represented in **Figure 2**, highlighting each stage involved in the system.

Figure 2: Workflow Diagram

Appendix B: Dataset and Annotation

The dataset consists of two classes: *Normal* and *Drowsy*. Sample images representing both classes are shown in **Figure 3**, demonstrating variations in driver states.

Figure 3: Sample Dataset Images (Normal vs Drowsy)

Each image is annotated using bounding boxes to define the region of interest. An example of annotation is shown in **Figure 4**.

Figure 4: Annotation Example (Bounding Boxes)

The dataset was prepared to ensure diversity in lighting, head pose, and facial expressions.

Appendix C: Data Processing and Augmentation

To improve model robustness, preprocessing and augmentation techniques were applied. The augmented outputs including flipping, brightness adjustment, and rotation are shown in **Figure 5**.

Figure 5: Augmented Images (Flip, Brightness, Rotation)

Appendix D: Model Architecture and Training

The architecture of the YOLO-based detection system is illustrated in **Figure 6**, showing how object detection is performed in a single forward pass.

Figure 6: YOLO Architecture Diagram

During training, model performance improved over epochs. Training behavior and metrics are visualized in **Figure 7**.

Figure 7: Training Graphs

The initialization and training setup of the YOLO model are shown in **Figure 8**.

Figure 8: YOLO Model Training and Initialization

Appendix E: Annotation and Implementation Tools

The annotation process and conversion into YOLO format are demonstrated in **Figure 9**.

Figure 9: Annotation and Conversion to YOLO Format

The dataset configuration used for training is shown in **Figure 10**.

Figure 10: Dataset Configuration Code

The model training implementation is provided in **Figure 11**.

Figure 11: Training Code

The real-time detection implementation using OpenCV is shown in **Figure 12**.

Figure 12: Prediction Code (OpenCV / Webcam)

Appendix F: Model Evaluation and Results

The evaluation of the model is based on standard metrics such as precision, recall, F1-score, and mAP.

The F1-score performance across thresholds is shown in **Figure 13**.

Figure 13: F1-Score Curve

Precision performance is illustrated in **Figure 14**.

Figure 14: Precision Curve

The trade-off between precision and recall is shown in **Figure 15**.

Figure 15: Precision–Recall Curve

The classification performance is further analyzed using confusion matrices. The normalized confusion matrix is shown in **Figure 16**, and the standard confusion matrix is shown in **Figure 17**.

Figure 16: Confusion Matrix (Normalized)
Figure 17: Confusion Matrix

Appendix G: Comparative Analysis

A comparison of different driver drowsiness detection approaches is presented in **Table X**.

Approach	Accuracy	Speed	Real-Time Capability	Limitations
EAR / PERCLOS	Medium	High	Limited	Sensitive to lighting, head movement, occlusion
Machine Learning	Medium	Medium	Limited	Requires manual feature extraction
CNN	High	Low	Limited	Computationally expensive
YOLO	High	Very High	Excellent	Slight localization trade-off

Appendix H: Supplementary Materials

The following materials support the implementation and reproducibility of the project:

- GitHub repository containing full project code
- Jupyter Notebook including training, evaluation, and inference code

Appendix I: Professional, Ethical, Social and Sustainability Considerations

This appendix provides supporting details for the responsible deployment of the proposed driver drowsiness detection system.

Professional Considerations:
The system must maintain high accuracy and reliability, as incorrect predictions may impact driver safety. Proper dataset preparation, annotation accuracy, and evaluation metrics (F1-score ≈ 0.99, mAP ≈ 0.995) ensure system robustness. Real-time testing using OpenCV further validates performance.

Ethical Considerations:
The system involves continuous monitoring using cameras, raising privacy concerns.

- User consent must be obtained before monitoring
- Facial data should not be stored unnecessarily
- System must comply with GDPR regulations
- Dataset diversity is important to reduce bias

Social Impact:
The system contributes positively by reducing accidents caused by driver fatigue and improving road safety. However:

- Users may feel uncomfortable with monitoring
- Over-reliance on automation should be avoided

Thus, the system should act as a **support tool**, not a replacement for driver responsibility.

Sustainability Considerations:
The YOLO-based system is computationally efficient:

- Single-stage detection reduces processing load
- Low latency supports real-time use
- Suitable for edge deployment

This leads to:

- Lower energy consumption
- Reduced hardware requirements
- Cost-effective implementation